A3 学生版讲义 · 从零补 Swing + 客户端 Socket,读者端 + 借书事务审查

> **课程**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元③ **用法**:课前预习、课后复习都能看。内容精炼,配合课堂消化。 **今天不手写业务代码**,今天**指挥 AI 写客户端,自己审查借书事务**。 **学情提示**:今天用到的 **Swing 图形界面** 和 **客户端 Socket** 你还没在课上学过——本讲义二~五节从零讲(配最小代码),先看懂再动手。看不懂的地方,当场让 AI 用中文逐行解释。

一、今天在整条路线里的位置

- **单元①**:反向提问 → 让 Trae 存 `.trae/rules`(图纸)。**图纸有了。**
- **单元②**:生成服务端,从零补 Socket + 线程,打通"登录"一条 C/S 链路。**服务端通了。**
- **单元③(今天)**:先从零补 **Swing 图形界面**和**客户端 Socket** 两块基础,再生成 **Swing 客户端 + 读者端**(查书 / 借书 / 还书),第一次真正的 C/S 联调,最后审 **借书事务**。

🔑 **C/S 心法**:登录那条链路一旦打通,后面查书 / 借书 / 还书都是**复制它换业务**——客户端拼一行请求 → 过 Socket 发给服务端 → 查库/处理 → 返一行 → 客户端解析、更新界面。**先做网络层 + 登录窗把界面连上服务端,别先堆界面。**

> 💬 一句话记住今天:单元② 补的是**服务端**那半边(等电话、开线程),今天补**客户端**这半边(拨号、拼界面)。补完这两块,C/S 的两头你就都懂了。

二、⭐ Swing 图形界面零基础:桌面界面的"积木"

你之前写的程序是"在黑控制台 `System.out.println` 打字";今天第一次做"有窗口能点鼠标"的程序(像 QQ / 浏览器)。Java 用 **Swing**(`javax.swing`,自带免装)做图形界面,窗口、按钮、表格都是现成"积木"拿来拼。

① **控件积木**(类比:**窗口是一块板子,控件是钉在板子上的零件**):

积木	类比	用在哪
`JFrame`	大板子(顶层窗口)	登录窗 / 读者主窗 / 管理员主窗
`JPanel`	板子上再钉的小分区板	搜索栏、按钮区等功能区块
`JLabel` / `JTextField`	说明纸条 / 填空格	"用户名:" / 输入框
`JButton`	按钮	登录 / 查询 / 借书 / 还书
`JTable`(配 `DefaultTableModel`)	表格纸	图书列表 / 我的借阅
`JComboBox`	下拉菜单	分类筛选下拉
`JTabbedPane` / `JDialog`	页签 / 弹出小板子	查询/我的借阅两页签、图书详情
`JOptionPane`	弹个小纸条	"借阅成功""请先选中一本书"

> 💬 不用背——记住"`JFrame` 是大板子,其它都是钉上去的零件"。看到 `new JButton("登录")` 就知道造了个写"登录"的按钮,`new JTable(...)` 就是造了张表格。

② **布局管理器 vs 绝对坐标**:别硬写坐标 `setBounds(x,y,w,h)`(窗口一缩放、换分辨率就乱、错位、被截断);用布局管理器让 Swing 自动算位置、随窗口自适应——`BorderLayout`(东西南北中,窗口整体骨架)/ `FlowLayout`(一排流式)/ `GridLayout`(规则网格)/ `GridBagLayout`(最灵活,登录表单对齐)。PRD 已写死"用布局管理器"。

> 审查信号:看到 `new BorderLayout()`、`add(表格, BorderLayout.CENTER)` = 对;一堆 `setBounds(...)` 写死坐标 = 提醒 AI 改回布局管理器。答案版登录窗用 `GridBagLayout` 让"服务器地址:"标签和输入框一行行对齐,主窗用 `BorderLayout` 把搜索栏放上、表格放中、按钮放下。

③ **事件与回调**(点了按钮为什么会执行代码):点按钮、双击行、输入框按回车叫**事件**;你把一段代码用 `addActionListener` "挂"到按钮上,平时不跑、**事件真的发生(按钮真被点)才被触发**——这叫**回调**(像**门铃**:提前接好线设好"响了播音乐",平时不响,有人按了才响)。这就是 GUI 和你以前程序最不一样的地方:**事件驱动**——程序先把界面摆好、把回调挂好,然后停着等你操作,点哪个按钮跑哪段回调。

```
JButton button = new JButton("点我");          // 造一个按钮
button.addActionListener(e -> {                 // 挂一段"被点时才执行"的代码(回调)
    System.out.println("按钮被点了!");          // 平时不跑,点了才跑
});
```

> 审查:看到 `loginBtn.addActionListener(e -> doLogin())`,就知道"点登录按钮 → 执行 doLogin()"。

④ **JTable + DefaultTableModel**(一批数据显示成表格):`JTable` 只管"怎么画表",数据由 `DefaultTableModel`(**数据罐子**)装着——往罐子装数据,表格自动显示;罐子里数据变了,表格跟着刷。每次查到一批书,清旧数据 + `addRow(...)` 新数据。审查只要认得出"表头列名 / 字段顺序对不对"即可。

```
// 数据罐子:第一个参数是列名(表头),0 = 初始 0 行
private final DefaultTableModel model = new DefaultTableModel(
        new Object[]{"ID","书名","作者","出版社","分类","库存"}, 0) {
    @Override public boolean isCellEditable(int r, int c) { return false; } // 不让直接改
};
private final JTable table = new JTable(model);    // 罐子交给表格,自动显示
```

⑤ **最小 GUI 长什么样**(实操 0 会让 AI 生成并跑通):

```
JFrame frame = new JFrame("我的第一个窗口");          // ① 造窗口
frame.setSize(320, 200); frame.setLocationRelativeTo(null); // 居中
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 点关闭就退出
JButton button = new JButton("点我");                  // ② 造按钮
button.addActionListener(e -> {                        // ③ 挂事件回调(点了才执行)
    JOptionPane.showMessageDialog(frame, "你点了按钮!现在是:" + new
java.util.Date());
});
frame.add(button, BorderLayout.CENTER);    // ④ 布局放到"中"区
frame.setVisible(true);                    // 显示窗口(不写这句什么都看不到)
```

> **GUI 全部骨架就这四步**:造窗口、放按钮、挂事件回调、显示。登录窗无非把"一个按钮"换成"几个输入框 + 登录按钮"、把"弹提示"换成"发 LOGIN 请求、按 role 开主窗"。今天客户端再复杂,骨架也是这四步。

三、⭐ 客户端 Socket 零基础:服务端的镜像 + net 三层

界面会拼了,点了按钮怎么把请求发到服务端?——用**客户端 Socket**。单元② 你学过服务端那半边(`ServerSocket` 守着等电话、`accept` 接);客户端就是**拨号方**,是服务端的**镜像**:

	服务端(单元②)	客户端(今天)
建连接	`new ServerSocket(9000)` 守着 + `accept()` 接	`new Socket("127.0.0.1", 9000)` 拨号
发一行	`write + "\n" + flush`(响应)	`write + "\n" + flush`(请求)
收一行	`readLine()` 收请求	`readLine()` 收响应
防中文乱码	读写流指定 UTF-8	读写流指定 UTF-8(**两头都要**,少一头就乱码)

心智模型不变:一条 socket 连接就是**一根能 `readLine()` 读一行、能 `write()` 写一行的管子**,客户端手里也是这根管子。

客户端**不直连数据库**(呼应档 B),只通过网络层和服务端说话。分三层:

```
client/ui          界面(登录窗 / 读者主窗 / 各面板;点按钮、显结果)
    │ 调用
client/net/Api     拼请求 → 发 → 解析响应(Api.login / Api.queryBooks / Api.borrow)
    │ 调用
client/net/SocketClient   真正的收发:发一行文本、收一行文本
    │ Socket
[服务端]
```

- **界面层**只管"点按钮调哪个方法、结果怎么显示",不碰协议字符串。
- **Api 层**负责拼协议(`LOGIN|用户名|密码`)、解析响应(`OK|数据` 拆数据、`ERR|原因` 抛异常)。
- **SocketClient** 最底层:和服务端保持一条长连接,`request(一行请求)` 返回一行响应。

SocketClient 三个关键设计(审查要看懂):

1. **长连接**:连一次一直不断开,反复用它发多条请求(查/借/还都走这一条);不每次都新建(建连接有开销)。
2. `synchronized` **一次一个请求**:否则两个界面同时发,响应可能**串行错位**(A 的请求收到 B 的响应)。它保证"发一行 → 收这一行的响应"是不可打断的整体。
3. **地址可配置** `configure(host,port)` + `getHost/getPort`,**不写死**——单机填 `127.0.0.1`、局域网(单元⑤)填服务器电脑 IP(如 `192.168.1.10`),登录窗的"服务器地址框"就填给它(**为局域网提前埋点**)。

```java
public static synchronized String request(String line) throws IOException {
    connect();                          // 没连就先连(长连接:已连直接用)
    out.write(line); out.write("\n"); out.flush();  // 发一行
    String resp = in.readLine();                    // 收一行
    return resp;
}
```

四、⭐ SwingWorker:网络请求不能"直接发"(子线程)

Swing 一条铁律:**所有界面更新都在同一个线程(EDT,事件分发线程)上做**,你点按钮触发的回调默认就跑在 EDT 上。而网络请求是**耗时**的(要等服务端查库、返回)——直接在点击回调里发,就是让 EDT 这个"唯一负责画界面的线程"干等网络,这段时间**整个界面卡死**(窗口不能动、按钮点不了、看着像死机)。

> 💬 类比:餐厅只有一个服务员(EDT),他既要上菜(画界面)又去后厨等菜(网络)——站后厨干等,前厅就没人管了。正解:把"等菜"交给**跑腿的(子线程)**,自己继续招呼客人,菜好了跑腿的回来喊他上菜。

解法:**SwingWorker**——耗时请求放子线程,跑完回 EDT 更新界面:

```
点按钮
  → SwingWorker.doInBackground()    子线程:发网络请求(界面此时仍能动)
  → SwingWorker.done()              回到 EDT:拿结果、刷新表格 / 弹提示
```

答案版把它封装成一行 `Async.run(父组件, 后台任务, 成功回调)`——后台线程发请求,成功回 EDT 更新界面、失败自动弹 `JOptionPane`。

```
// 父组件，后台任务(发网络请求)，成功回调(回到 EDT 更新界面)
Async.run(this, () -> Api.login(username, password), user -> { /* 登录成功后打开主窗
*/ });
```

> 信号:**AI 生成的界面点一下就卡死、转圈半天** → 九成是请求直接写进了点击回调、没放 SwingWorker(或 Async)。这是今天联调最常见的坑之一,也是你要审的:凡是发网络请求的按钮,请求都得在子线程里发。

五、⭐ 代码审查:AI 也会犯错

前面单元你会"让 AI 写";今天开始要会"审 AI 写的"——尤其借书这种涉及事务、并发的关键代码。

AI 写 Swing 界面、Socket 收发这类**样板活**又快又基本靠谱,但一旦涉及**事务、并发、业务一致性**,它会:

- **图省事**:借书"先查库存够不够,再减"分两步——单机看着对,并发下**超借**(两人同时查到剩 1 都借走)。
- **漏关键约束**:减库存忘了 `WHERE stock>0`、忘了判影响行数、减库存和写记录**没放一个事务**。
- **看起来能跑**:单机一个人点全对——**但并发一致性的坑,单机测不出来**。

铁律:信任但验证。 AI 写的代码不能照单全收,涉及事务 / 并发 / 安全的必须审。这不是不信任 AI,而是**分清哪些能放过、哪些必须盯**:放过该盯的 → 上线出事(超借、数据不一致);每一行都手抠 → 又失去了 Vibe Coding 的速度。怎么分?看**信任度金字塔**:

层级	例子(我们的图书馆)	怎么对待
🟢 高信任	界面布局、样式、JTable 展示、图书查询(纯读)	扫一眼、跑通即可
🟡 中信任	业务逻辑:借阅上限校验、还书状态流转、字段映射	核对逻辑对不对、边界有没有漏
🔴 必人工	**事务 + 并发一致性**:借书减库存 + 写记录同一事务、防超借;安全:密码、注入	**逐条审,你说了算**

今天的重头就在 🔴 这一档:**借书事务**。单元① 你在 PRD 里写死的规则,现在就是审查的**验收标准**。

> 💬 **审查三个动作**:① **对照 PRD**(有没有 `setAutoCommit(false)` / `commit` / `rollback`、有没有带 `AND stock>0` 的 UPDATE);② **看懂逻辑**(看不懂让 AI 中文逐行解释);③ **验证行为**(单机不好测,单元⑤ 局域网开两个客户端同时借最后一本,验证只成一个)。发现不对**不自己改**,用纠错三步法"只改这一处"让 AI 补。**规范是新的源代码,审查就是拿规范去验代码;AI 审代码是助手,拍板的是你。**

六、SOLO prompt 速查(可复制)

Step 0 · 最小 Swing GUI(热身,亲眼看窗口和事件):

用纯 Java 11 Swing 写一个最小的图形界面 demo,一个可直接运行的类 HelloSwing:一个 JFrame 窗口(标题"我的第一个窗口",320x200,居中,点关闭退出);中间一个 JButton"点我";给按钮加点击事件,点了弹 JOptionPane 显示一句话 + 当前时间;布局用 BorderLayout,不写死坐标。只要这一个类,能直接编译运行。

Step 1 · 网络层 SocketClient + 登录窗(含服务器地址框):

帮我创建 Java Swing 客户端:
- client/net/SocketClient:与服务端 127.0.0.1:9000 建长连接,提供 request(String) 发一行收一行(synchronized);host/port 做成可配置 configure(host,port)+getHost/getPort,不要写死。
- client/ui/LoginFrame:登录窗,顶部有"服务器地址""端口"两个输入框(默认 127.0.0.1 / 9000),下面用户名、密码;登录按钮点了先按输入框 configure 再发 LOGIN,网络请求放 SwingWorker,失败弹 JOptionPane;成功按 role 打开读者主窗或管理员主窗。布局用布局管理器。
先做网络层 + 登录窗,主窗先放空壳。

Step 2 · 图书查询 + 详情 → 借书:

基于现有代码,做读者端:服务端加 BOOK_QUERY|关键字|分类id(BookService/BookDao,LEFT JOIN 分类带出 categoryName,Json 编码返回)、CATEGORY_LIST;客户端读者主窗:搜索栏(关键词+分类下拉)+ JTable 图书列表,双击一行弹详情对话框(带"借书"按钮)。先做查询和详情展示,借书下一步。只加图书查询相关。

基于现有代码,加借书:协议 BORROW|userId|bookId;服务端 BorrowService.borrow 在一个事务里:UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0(按影响行数判库存)、校验读者未还数<max_borrow、插入 t_borrow(due=今+30天),成功 commit 返 OK|应还日期,失败 rollback 返 ERR。客户端详情"借书"按钮调 BORROW,提示结果并刷新库存。只做借书。

Step 3 · 我的借阅 + 还书:

基于现有代码,加"我的借阅":协议 MY_BORROW|userId(BorrowDao LEFT JOIN 出书名/借出/应还/状态)、RETURN|borrowId(事务:标 returned+库存+1,防重复归还)、RENEW|borrowId;客户端读者主窗加"我的借阅"选项卡:JTable + 每行"还书/续借"按钮。只做借阅记录和还书。

七、读者端四层职责表(看懂生成的代码)

层	类	干什么	一句话
界面	`LoginFrame` / `ReaderMainFrame` / `BookQueryPanel` / `BookDetailDialog` / `MyBorrowPanel`	摆控件、挂事件回调、显结果	只调 `Api.*`,不碰协议字符串、不碰库
后台	`Async` (封装 SwingWorker)	把耗时请求放子线程,成功回 EDT	界面不卡死的关键
接口	`client/net/Api`	拼协议、解析响应	`Api.login` / `queryBooks` / `borrow` / `returnBook`
收发	`client/net/SocketClient`	长连接 + `synchronized` 发一行收一行	地址可配置、UTF-8、客户端唯一出网口

> 读者主窗用 `JTabbedPane` 装两个页签:`addTab("图书查询", new BookQueryPanel())`、`addTab("我的借阅", new MyBorrowPanel())`——这就是第二节那块"页签"积木。

八、⭐ 借书事务审查清单(逐条判断"AI 写对了吗")

#	审查点(PRD 要求)	对的样子
1	是不是一个事务?	`setAutoCommit(false)` → `commit()` / `rollback()` 三件套
2	减库存 `WHERE stock>0` ?	`UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0`
3	按影响行数判库存?	`affected==0` 就 rollback 报"库存不足"(不是先 SELECT)
4	减库存+写记录同一事务?	两步用同一 `conn`,一起 commit
5	借阅上限校验?	`countUnreturned >= maxBorrow` 就 rollback(🟡 中信任)

✅ **五条都对** = AI 这次借书写对了,信任但已验证,放行。

AI 三种埋坑写法(单机一个人点全对,坑藏在并发里,必须审):

- ① **先 SELECT stock 再 UPDATE**(不带 `stock>0`)→ 并发超借(两人同时查到剩 1 都借走)。
- ② **减库存和写记录不在一个事务**(分两次拿连接)→ 可能减了库存却没写成记录,数据对不上。
- ③ **UPDATE 了不判影响行数** → 库存 0 时没减到行却照样写记录,凭空多借一本。

发现了用**纠错三步法**(定位 → 描述 → 只改一处)让 AI 补,例如:"借书扣库存不要先 SELECT 再 UPDATE,改成 `UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0`,按影响行数是否为 0 判断库存不足。只改这一处。"

> 还书是**同一套路**:`WHERE id=? AND status='borrowed'` 按影响行数判断——影响行数为 0 说明"已经还过了",**防重复归还**把库存越加越多。

八·补 · 续借规则 + 逾期罚款(prompt 速查 + 审查要点)

> 这两块是 Step 3 之后追加的实操:续借练"审出答案版都漏的规则",罚款练"涉钱必人工"。**能点 ≠ 规则对;拿业务规则去照代码,才是审查。**

① 续借 RENEW · prompt:

```
基于现有代码,完善读者端"续借"(RENEW):协议 RENEW|borrowId;BorrowService.renew 只有当记录是 borrowed 状态才允许续借,用 UPDATE ... SET due_time = DATE_ADD(due_time, INTERVAL 30 DAY) WHERE id=? AND status='borrowed' 往后延 30 天,按影响行数判断,影响行数为 0 返回 ERR;成功 SELECT 新 due_time 以 OK|新应还日期 返回;客户端"我的借阅""续借"按钮调 RENEW,成功弹新应还日期并刷新。只改续借相关。
```

⭐ **续借审查要点**(答案版自己都漏了一条,自己审出来):

#	续借规则	对的样子
1	延多少天	`RENEW_DAYS=30`,从旧 `due_time` DATE_ADD,不是从今天算
2	已还能续吗	`WHERE id=? AND status='borrowed'`,returned 匹配不到
3	怎么知道续失败	`affected==0` 抛错返 ERR
4 🔴	**续借次数上限**	**答案版漏了!** 任何 borrowed 记录都能一续再续

补齐(和借书 `WHERE stock>0` 同一套心法,一句原子 UPDATE 把"限次数 + 记一笔"做掉):

```
ALTER TABLE t_borrow ADD COLUMN renew_count INT NOT NULL DEFAULT 0;
UPDATE t_borrow SET due_time = DATE_ADD(due_time, INTERVAL ? DAY), renew_count =
renew_count + 1
WHERE id=? AND status='borrowed' AND renew_count < 1;   -- affected==0 → 不存在/已
还/超限
```

> ⚠️ 最隐蔽的错:判了 `renew_count < 1` 却忘了 `renew_count + 1`——第一次没自增、第二次照样能续。**必须验到"第二次续借被拒"才算通过。**

② 逾期罚款 · 强 prompt(规则说死,答案版无真码走 prompt 模式):

```
给还书流程加"逾期罚款",严格照做:每逾期 1 天罚 0.5 元,逾期天数=实际归还时间与 due_time 的天数
差向上取整;只在晚于 due_time 时才罚,没逾期不写任何记录;计算写入放进还书事务、同一个
Connection 一起 commit,逾期就 INSERT INTO t_fine(borrow_id, amount, paid)
VALUES(?,?,0);金额一律用 BigDecimal/DECIMAL,禁止 double/float 算钱;还书返回值带罚款金
额,客户端提示给用户。只改还书相关。
```

⭐ **逾期罚款审查要点**(涉钱必人工,复用借书那把尺子):

#	审查点	怎么算对	信任档
1	日期差拿谁比谁	实际归还时间 vs `due_time`,不是借出时间 `borrow_time`	🟡
2	没逾期会误罚吗	逾期天数 ≤ 0 一条罚款都不写(不是写 0 元)	🟡
3	金额精度	用 `BigDecimal`/`DECIMAL`,**不用 double**(`0.5*3` 浮点会出 `1.5000000002`)	🔴 涉钱
4	罚款和还书一个事务吗	INSERT t_fine 和还书 UPDATE 同一 conn、一起 commit	🔴
5	天数取整	按 PRD 向上取整,不确定就问 AI 用没用 ceil	🟡

> 💬 涉钱一律 🔴 必人工——AI 爱写 `double amount = 0.5 * days`,单机看着没事,金额一累加浮点误差就冒出来,对账差几分钱说不清。**造数据验**(单机可测):MySQL 里 `UPDATE t_borrow SET due_time='2020-01-01 00:00:00' WHERE id=…` 造出逾期再还,查 `t_fine` 有没有生成、金额=逾期天数×0.5 对不对;正常借一本立刻还,`t_fine` 不应新增。

九、联调三坑速查

现象	原因 → 怎么修
连不上 / Connection refused	服务端没起(先起单元② 服务端)/ 端口不对(都填 9000)/ 地址填错(单机填 127.0.0.1)
点登录界面卡死、转圈	请求没放 SwingWorker、占住 EDT → "登录请求放进 doInBackground,别在点击事件里直接发。只改这一处"
中文书名 / 姓名乱码	收发流没 UTF-8 → "InputStreamReader/OutputStreamWriter 都指定 StandardCharsets.UTF_8。只改这一处"

十、读者端一条龙(今天的验收)

```
登录 zhangsan/123456
  → 图书查询:看到种子图书(带分类名、库存),输关键字能筛选,双击弹详情
    → 借一本(详情点"借这本书")→ 提示"借阅成功!应还日期:…" → 列表该书库存 -1
      → 切到"我的借阅":看到这条记录(状态"借阅中")
        → 选中点"还书"→ 提示成功 → 记录变"已归还"
          → 回图书查询,该书库存 +1 回来
```

三段都跑通 + 审过借书事务 = 今天目标达成:**客户端由 AI 施工、你把它联调跑通、并审查了最关键的借书事务。**

十一、FAQ

Q0:Swing 我一点没学过,今天能跟上吗? 能。今天不要求你会写 Swing 语法,只要看懂"界面是哪些积木拼的、点了按钮会发生什么"。记住那个类比:**窗口是板子,控件是钉在板子上的零件**;点按钮触发挂上去的回调(像门铃)。实操 0 会让你亲手跑通一个最小窗口先建立实感。

Q1:全程让 AI 写界面,我岂不是不会写 Swing? 你练的是更值钱的能力:看懂界面对不对(用没用布局管理器、请求有没有放 SwingWorker)、审查关键的借书事务。AI 写得快,但对不对你判断——这是你的核心竞争力。

Q1.5:为什么点了按钮才执行代码?我以前的程序都是从上往下跑的。 这就是 GUI 的**事件驱动**:程序先把界面摆好、把各种回调"挂"到按钮上,然后停着等你操作;你点哪个按钮,挂在那个按钮上的回调(`addActionListener` 里那段)才被触发执行。和你以前"从上往下一行行跑到底"的控制台程序不一样。

Q2:客户端 Socket 和单元② 的服务端 Socket 有什么关系? 是**镜像**。服务端 `accept()` 接电话,客户端 `new Socket(ip,port)` 拨电话;之后"发一行、收一行、UTF-8"两头一模一样。心智模型也一样:一条 socket 连接就是一根能读一行、写一行的管子。你其实已经会一半了。

Q3:SocketClient 为什么要长连接、还要 `synchronized`? 长连接:连一次一直用,查/借/还都走这一条,省掉每次新建连接的开销。`synchronized`:保证同一时刻只有一个请求在管子上收发,否则两个界面同时发,响应可能串行错位(A 的请求收到 B 的响应)。

Q4: `Async.run` 和 SwingWorker 是一回事吗? 是。`Async.run(父组件, 后台任务, 成功回调)` 就是答案版把 SwingWorker 那套"子线程发请求 → 回 EDT 更新界面 → 失败弹窗"封装成的一行。如果 Trae 直接用了原生 `SwingWorker` 也对,原理一样:耗时请求放子线程、别占住 EDT。

Q5:借书单机点着都好好的,为什么还要审? 因为**跑通 ≠ 正确**。借书涉及并发,单机一个人点测不出超借的坑。三种埋坑写法单机全对,只有并发下(两人同时借最后一本)才暴露。必须拿 PRD 一条条量。

Q6:什么代码要审、什么可以放过? 看信任度金字塔:🟢 界面/纯查询扫一眼跑通即可;🟡 业务逻辑核对逻辑;🔴 事务/并发/安全(借书事务是典型)逐条审。别把 🔴 当 🟢 放过了。

Q7:界面点一下就卡死怎么办? 九成是网络请求直接写在了点击事件里、没放 SwingWorker(或 Async)。让 AI 把请求放进 `doInBackground`,成功回调里再更新界面。

Q8:防超借为什么用 `WHERE stock>0` 而不是先查再减? `UPDATE ... WHERE id=? AND stock>0` 靠数据库行锁保证原子性:多个客户端并发借同一本,只有一个能让 stock 从 1 变 0,其余影响行数为 0 被判"库存不足",绝不超借。先 SELECT 再 UPDATE 是两步、中间有空隙,并发下会超借。

今日产出

从零补上 **Swing 图形界面**(GUI/窗口/控件积木/布局/事件回调/JTable+DefaultTableModel/SwingWorker)和 **客户端 Socket**(拨号连服务端对照服务端/net 三层/发一行收一行/长连接+synchronized+可配置),亲手跑通一个最小 Swing GUI;再用 SOLO **分步生成客户端**(网络层 + 登录窗 → 查书 + 详情 → 借书 → 我的借阅 + 还书),完成第一次 **C/S 联调**(读者端一条龙:查 / 借 / 看 / 还,库存实时增减);并对 **借书事务**做了一轮**信任但验证**(对照 PRD 五条清单逐条判断、认得出三种埋坑写法)。**AI 施工,你验收——你手里多了一双审 AI 关键代码的眼睛。** 下个单元做管理员端 + 服务端监控窗口,今天补的 Swing、练的审查继续用。